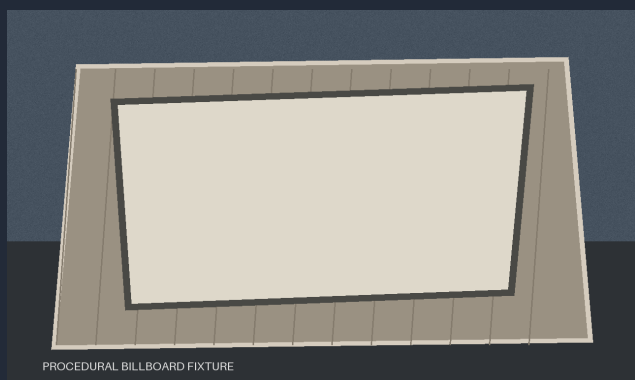
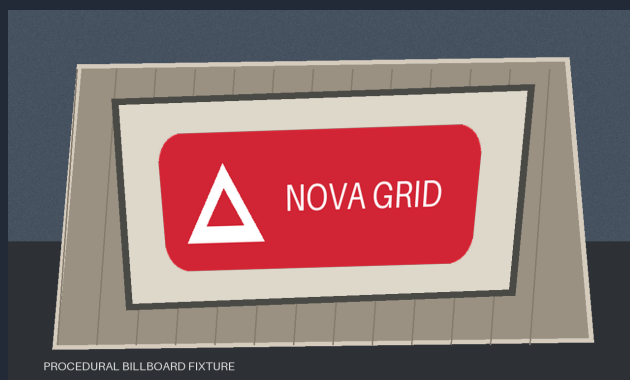


Exact brand in. Realistic mockup out.

SponsorSkin combines deterministic computer vision, masked generative refinement, and exact-logo restoration for auditable sponsorship mockups.



ORIGINAL



LOCAL ROUGH COMPOSITE

EXACT

logo geometry retained

LOCAL

editable region isolated

AUDITABLE

artifacts + manifests

LOCAL DEVELOPMENT CANDIDATE - RADEON VALIDATION PENDING

Brand-safe ideation without the manual mockup bottleneck

The problem

Generic generators can produce attractive concepts, but often rewrite lettering, alter proportions, move symbols, or shift brand colors. Traditional mockups preserve the asset but cost time and specialist effort.

SponsorSkin divides the job: computer vision owns logo identity; a masked model owns only local material, lighting, reflection, and texture.

Target users

- Freelance and in-house designers
- Small motorsport and event teams
- Brand, venue, and signage planners
- Apparel creators and fabric printers
- Small businesses planning fabrication

APPLICATION SCENARIOS

BILLBOARD

Signage concepts before media production.

UPLOAD + 4 CLICKS

preview, compare, export

VEHICLE

Sponsor inventory and vinyl placement.

UPLOAD + 4 CLICKS

preview, compare, export

FABRIC

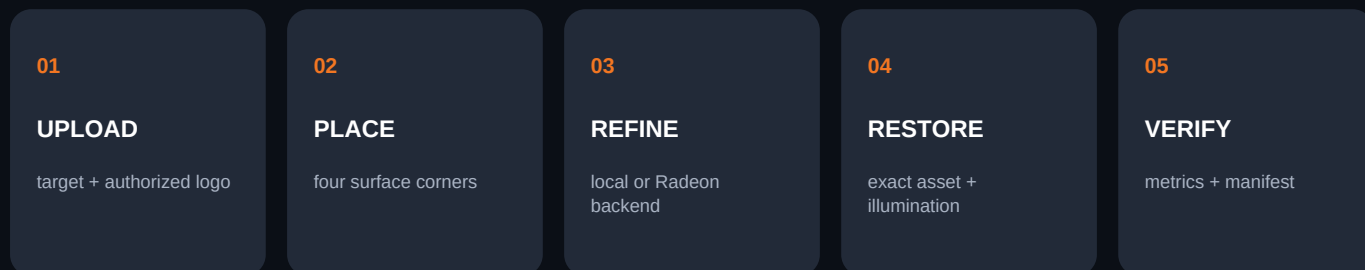
Print scale and position on apparel.

UPLOAD + 4 CLICKS

preview, compare, export

PRACTICAL VALUE: FASTER EARLY CREATIVE DECISIONS

One interface, explicit boundaries, reproducible outputs



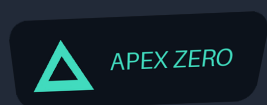
System invariants

- Inputs are never overwritten.
- Exact undilated alpha is retained.
- Black mask pixels return to source.
- Passthrough is pixel-idempotent.
- Radeon mode fails closed without ROCm.
- Every run gets a unique manifest.

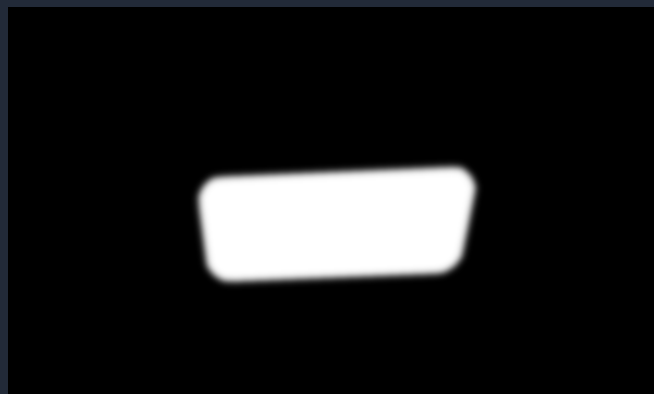
Design decision

The refinement backend can change without duplicating validation, geometry, brand restoration, metrics, or evidence logic.

Deterministic placement before and after generation



EXACT RGBA LAYER



FEATHERED EDIT MASK

Geometry and masking

- Validate safe SVG or transparent PNG and preserve alpha.
- Order four clicks into a convex, non-self-intersecting quadrilateral.
- Compute an OpenCV perspective homography and warp exact RGBA pixels.
- Keep undilated exact alpha for restoration; dilate and feather only the edit mask.

Exact-logo restoration

Estimate the refined-to-rough luminance ratio inside stable logo pixels in linear RGB. Clamp and smooth that illumination field, apply it to the exact warped logo, composite with original alpha geometry, then lock every pixel outside the edit mask back to the source.

A conservative masked path designed for measurable Radeon execution

IMPLEMENTED

FLUX.2 Klein 4B inpaint backend

The local rough composite is the source image and the feathered mask limits the editable region. Four material presets cover vinyl, fabric print, billboard, and painted wall.

DTYPE

BF16

STEPS

4 initial

STRENGTH

0.65

GUIDANCE

1.0

MAX AREA

1,048,576 px

ROCm contract

- Retain the platform ROCm PyTorch build.
- Verify torch.version.hip and device.
- Use the torch.cuda compatibility API.
- Start with standard PyTorch attention.
- Avoid CUDA-only extension assumptions.
- Record device and dependency revisions.

Cloud acceptance gate

- Doctor confirms real ROCm device.
- Smoke test completes one masked edit.
- Outside-mask preservation remains exact.
- Three warm runs are stable.
- Latency and peak VRAM are raw JSON.
- Model revision is pinned after success.

PENDING REAL RADEON CLOUD EXECUTION

No local macOS result is presented as model inference or AMD GPU performance.

Every claim maps to an artifact, metric, or explicit pending gate

35 / 35

CPU-safe tests passing

0.000

outside changed ratio

1.000

outside SSIM

Run artifact contract

- original and source logo
- exact warped RGBA layer
- rough composite and two masks
- refined and final images
- illumination visualization
- metrics.json and manifest.json

Measured local path

0.425 s

WARM MEAN - 1280 x 768 - 4 WARM RUNS

Complete deterministic passthrough artifact path on Apple M3 Pro. This measurement contains no generative model and no Radeon GPU.

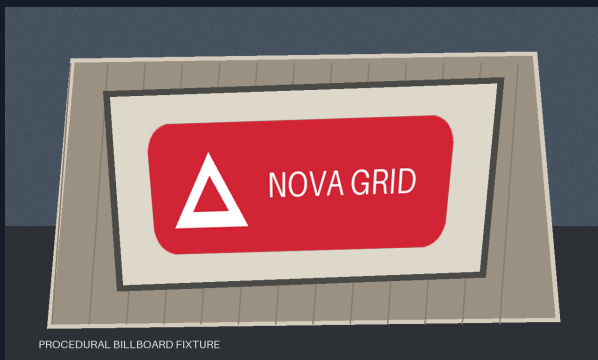
LOCAL CPU EVIDENCE

Automated signals

Outside changed-pixel ratio / outside SSIM and mean absolute error / logo Delta E 2000 / variance-of-Laplacian sharpness / mask coverage / warnings / latency / environment provenance

RADEON LATENCY + VRAM PLACEHOLDER: CLOUD RUN REQUIRED

Fictional procedural fixtures make local behavior reproducible and rights-clear

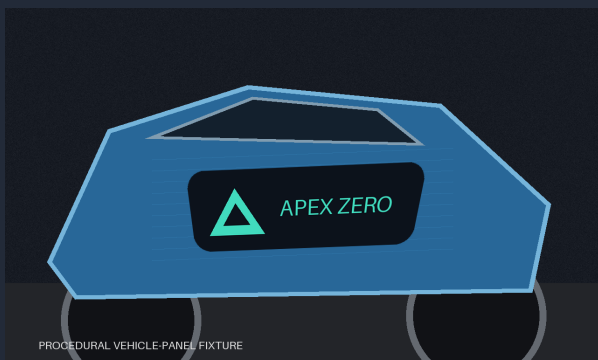


NOVA GRID / BILLBOARD

Committed input, fictional transparent logo, four corners, exact layer, mask, and deterministic rough preview.

LOCAL PASSTHROUGH

billboard



APEX ZERO / VEHICLE VINYL

Committed input, fictional transparent logo, four corners, exact layer, mask, and deterministic rough preview.

LOCAL PASSTHROUGH

vehicle-panel



KINETIQ / FABRIC PRINT

Committed input, fictional transparent logo, four corners, exact layer, mask, and deterministic rough preview.

LOCAL PASSTHROUGH

fabric

Final judging examples should add rights-cleared real photographs and real Radeon refinements while retaining these fixtures as regression evidence.

Local package ready; cloud evidence deliberately unfilled

Ready locally

- Deterministic CV stack
- Exact-logo restoration
- Metrics and manifests
- Interactive Gradio app
- Radeon backend contract
- Doctor and benchmark tools
- Three demo fixtures
- 35 passing tests

Radeon Cloud gate

- Confirm ROCm environment
- Download and pin model revision
- Run masked smoke test
- Capture cold and warm timing
- Capture peak allocated VRAM
- Record real GPU and ROCm facts
- Export actual refined examples
- Record 3-5 minute demo video

Known limitations

Current examples are procedural development fixtures, not real photographs. Local outputs are deterministic rough previews, not model refinements. Radeon compatibility, visual improvement, latency, and peak VRAM require real cloud execution. LoRA remains an optional experiment.

REFERENCES

luma.com/amd-4dhi | huggingface.co/black-forest-labs/FLUX.2-klein-4B | huggingface.co/docs/diffusers/api/pipelines/flux2 | github.com/JaviChulvi/Radeon-hackathon-2026-07

LOCAL DEVELOPMENT CANDIDATE - RADEON VALIDATION PENDING